# Using Reinforcement Learning to Learn Search Strategies for Black-Box Optimization, with an Application to Honeypot Search

CS 4180/5180 Project Proposal · Aniket Ghosh · Individual project

## 1. Problem description

**What problem are you solving?**

- I will train an RL agent to act as an optimizer. It will choose which setting to try next when each try is slow and noisy, like hyperparameter tuning.
- I will apply it to honeypot search. A honeypot is a gridworld level and PPO settings where the PPO agent takes a planted shortcut (a "grader" square) instead of doing the real task [2].

**Describe the problem formally from a computational perspective.**

- **Episode:** each episode will be one search on a function $f$: $[0,1]^n \to \mathbf{R}$, with a budget of $B$ tries ($B$ = 200–10,000).
- **Step:** the agent will pick a point, I will measure $f$ there, and the result will go into the search history. Results will be noisy and some points will be invalid.
- **Discount factor:** I will use 1 for ES and 0.995 for PPO and DQN.

**What are the states, actions and rewards?**

- **State:** the state will be the list of hyperparameter values. **Action:** the action will change a hyperparameter (for example, +1 to a numerical hyperparameter). Both will depend on the agent (Table 1).
- **Reward, option 1 (sparse):** $-\log_{10}$(final regret / value of a random point), given once at the end. I will use it for ES.
- **Reward, option 2 (dense):** the log of the improvement, given every step. I will use it for PPO and DQN, and I will also train them with option 1 to compare the two.

| Agent | State (features of the search history) | Actions |
|---|---|---|
| **Walker** | Per coordinate (K = 8: last move, momentum axis, position, offset to best / step, …) and global (G = 6–7: success-rate EMA, log step, budget fraction *t/B*, momentum norm, is-best flag, repeats at best). | Per coordinate {−1, 0, +1}; global heads for step multiplier in {0.5, …, 8}, momentum β, pull-to-best γ, subspace κ; extra actions {move, re-check best, jump}. |
| **RL-CMA** | 7 features of the CMA-ES run: σ ratio, budget fraction, improved, stall, population spread, popsize. | σ multiplier; restart. |
| **rlbo** | 20 features per candidate, from a local GP ensemble and a global quadratic surrogate. | Pick 1 of 48 candidates from 6 generators; trust radius ×{0.5, 1, 2}. |

*Table 1. States and actions for my three optimizers.*

**What simulator or domain are you using (exactly)?**

- **MetaWalker-Bench** (my own benchmark): I will use standard test functions [22, 23], rotated and shifted as in BBOB [22]. I will train on sphere, ellipsoid, Rastrigin and Rosenbrock, and test on Ackley and Levy. I will also add a flat "honeypot-like" family (95–99% flat, 3 hidden directions, binomial noise). $n$ = 5–500.
- **Real honeypot:** I will run PPO in JAX (PureJaxRL style [11]) on a 13×17 grid: get the key, open the door, reach the goal. A grader square is the shortcut.
- **What I will search over:** 169 numbers: 154 wall genes; 9 position genes (x and y for the start, key, grader and goal, plus the door's row); 1 grader-visibility gene (which I will fix to hidden in the hardened version I report); 5 PPO hyperparameters.
- **Score:** I will measure how often PPO takes the shortcut, over 5 seeds. A level will only count if the goal is reachable (BFS), it passes a distance check, and an honest version is still learnable (score ≥ 0.7).

**Why is it interesting?**

- Popular tuners like GP-based Bayesian optimization [18] and TPE [3] get slow and weaker as the number of parameters grows.
- Honeypot problems are mostly flat, noisy and full of invalid points. These tuners struggle there.

**Does it already have an interface for RL?**

- No Gym environment exists. I will use a simple ask/evaluate loop (`AskTell`) from my Python package `metawalker`, which can be installed with pip from the repository. I will also publish it on PyPI.

**Have other researchers already used it?**

- The test functions are standard [22, 23]. Others have trained optimizers before: MetaBO [21], MetaBox [12], learned ES [8], RL control of CMA-ES step size [16].
- Reward-hacking testbeds exist for LLM agents: BaitBench [17], Countdown-Code [7], "Winning at All Cost" [13].
- I have not found anyone using a learned optimizer to find honeypots, and I will finish checking before the final report.

## 2. Algorithms

**What algorithms do you use (exactly)?**

- **[New] Walker:** I will build a small neural network, shared across coordinates, that moves the search point, and train it with OpenAI-ES [14].
- **[New] RL-controlled CMA-ES:** I will build a policy that sets the CMA-ES step size and decides when to restart, on top of IPOP-CMA-ES [6].
- **[New] rlbo:** I will build a policy that picks the next point from 48 candidates using surrogate-model features (same idea as MetaBO [21]), and train it with ES.
- **[Existing] PPO** [15] and **double DQN** [20] with branching [19]. The algorithms exist; I will train them on this problem, which is new.
- I am working alone, so I will do all four.

**What baseline methods will you compare to?**

- **[Baseline]** I will compare against CMA-ES (IPOP, pycma [24]); GP-BO and TPE from Optuna [1]; SMAC3 [10]; Nevergrad NGOpt [25]; random search; simple hand-written walkers (1/5th-success rule).

| Part | Existing (from other people) | New (my addition) |
|---|---|---|
| **Optimizers** | CMA-ES with IPOP restarts [6]; ES training [14]; PPO [15]; double DQN [20] with branching [19]; the ideas of RL step-size control [16] and learned point picking [21]. | The walker policy; the RL controller for CMA-ES (step size and restarts); rlbo's features and candidate generators; training all of them on one shared MDP. |
| **Problems** | Standard test functions (sphere, ellipsoid, Rastrigin, Rosenbrock, Ackley, Levy) [22, 23]; PureJaxRL-style PPO [11]. | The flat "honeypot-like" test family; the key → door → goal honeypot gridworld and its validity checks; the `metawalker` package. |
| **Baselines** | All of them: CMA-ES, GP-BO, TPE, SMAC3, NGOpt, random search, 1/5th-rule walkers. | None. |
| **Use case** | Reward-hacking testbeds for LLM agents [7, 13, 17]. | Using a learned optimizer to search for honeypots. |

*Table 2. What I take from others and what I add.*

**Why are these algorithms appropriate?**

- An optimizer makes a chain of decisions with a fixed budget. That is an RL problem.
- ES copes with a reward that is sparse and noisy, because it does not need gradients.
- PPO and DQN will check whether standard RL can learn the same thing.

**How are these algorithms typically used, and how are you using them?**

- **Usually:** ES and PPO train agents that control something, like a robot or a game player.
- **Here:** I will use them to train a search strategy on many functions. I will then run it, without retraining, on new functions and on the real honeypot.

**Have other people used similar algorithms to solve your problem before?**

- For general optimization, yes [4, 5, 8, 9, 16, 21]. For honeypot search, not that I know of.

## 3. Results

**What results do you expect to show?**

- I expect my optimizers to do as well as or better than the baselines with the same number of tries, and to run much faster than GP.
- I expect them to find stronger honeypots than the baselines with the same PPO compute.
- I expect the ablations to show which learned actions matter (for example, the walker's jump and re-check moves).

**What comparisons will you do?**

- I will compare against GP, CMA-ES and TPE with the same number of tries, for $n$ = 5–500.
- I will compare against GP with the same running time. I will also compare RL-CMA with plain CMA-ES, to show the RL part itself helps.
- I will test budgets from 300 to 10,000 tries.
- I will run every method on the real honeypot with the same PPO compute.
- I will run ablations that remove one learned action at a time, and I will look at what the policies learned.

**Are there risks for not getting all the results? If so, what will you do about it?**

- **Noisy scores:** the best results can be lucky. I will re-test them on new seeds.
- **Bad PPO seeds:** some seeds may never learn the honest task, which would add noise. A check will filter them out, and I will report how often it happens.
- **Compute:** I will run on the CPU-only Explorer cluster. If honeypot runs are too slow, I will keep a 200-try honeypot test and use the cheap flat family for the rest.
- **Bugs:** things like seed leaks or tied ES ranks could bias results. I will use self-checks and held-out seeds.
- **Few seeds:** I will report Wilson confidence intervals.
- **PPO or DQN may fail:** the ES agents will not depend on them, so a failure will still be a result I can report.

## LLM-assistance disclosure

I will write all the base algorithms myself. Claude (Claude Code) will write the shell scripts and other supporting code, including the experiment scripts in `experiments/` (`experiments/honeypot/` and `experiments/analysis/`). I know this means a higher grading standard, and I will be ready to explain every algorithm and result.

## References


[1] Akiba, T., Sano, S., Yanase, T., Ohta, T., & Koyama, M. (2019). Optuna: A next-generation hyperparameter optimization framework. *KDD*.

[2] Amodei, D., Olah, C., Steinhardt, J., Christiano, P., Schulman, J., & Mané, D. (2016). Concrete problems in AI safety. arXiv:1606.06565.

[3] Bergstra, J., Bardenet, R., Bengio, Y., & Kégl, B. (2011). Algorithms for hyper-parameter optimization. *NeurIPS*.

[4] Chen, Y., Hoffman, M. W., Colmenarejo, S. G., Denil, M., Lillicrap, T. P., Botvinick, M., & de Freitas, N. (2017). Learning to learn without gradient descent by gradient descent. *ICML*.

[5] Gomes, H. S., Léger, B., & Gagné, C. (2021). Meta learning black-box population-based optimizers. arXiv:2103.03526.

[6] Hansen, N., & Ostermeier, A. (2001). Completely derandomized self-adaptation in evolution strategies. *Evolutionary Computation*, 9(2), 159–195.

[7] Khalifa, M., Khan, Z., Tafveez, O., Peng, H., & Wang, L. (2026). Countdown-Code: A testbed for studying the emergence and generalization of reward hacking in RLVR. arXiv:2603.07084.

[8] Lange, R. T., Schaul, T., Chen, Y., Zahavy, T., Dalibard, V., Lu, C., Singh, S., & Flennerhag, S. (2023). Discovering evolution strategies via meta-black-box optimization. *ICLR*.

[9] Li, K., & Malik, J. (2017). Learning to optimize. *ICLR*.

[10] Lindauer, M., Eggensperger, K., Feurer, M., Biedenkapp, A., Deng, D., Benjamins, C., Ruhkopf, T., Sass, R., & Hutter, F. (2022). SMAC3: A versatile Bayesian optimization package for hyperparameter optimization. *JMLR*, 23(54).

[11] Lu, C., Kuba, J. G., Letcher, A., Metz, L., Schroeder de Witt, C., & Foerster, J. (2022). Discovered policy optimisation. *NeurIPS*. (PureJaxRL)

[12] Ma, Z., Guo, H., Chen, J., Li, Z., Peng, G., Gong, Y.-J., Ma, Y., & Cao, Z. (2023). MetaBox: A benchmark platform for meta-black-box optimization with reinforcement learning. *NeurIPS Datasets & Benchmarks*. arXiv:2310.08252.

[13] Malmqvist, L. (2025). Winning at all cost: A small environment for eliciting specification gaming behaviors in large language models. arXiv:2505.07846.

[14] Salimans, T., Ho, J., Chen, X., Sidor, S., & Sutskever, I. (2017). Evolution strategies as a scalable alternative to reinforcement learning. arXiv:1703.03864.

[15] Schulman, J., Wolski, F., Dhariwal, P., Radford, A., & Klimov, O. (2017). Proximal policy optimization algorithms. arXiv:1707.06347.

[16] Shala, G., Biedenkapp, A., Awad, N., Adriaensen, S., Lindauer, M., & Hutter, F. (2020). Learning step-size adaptation in CMA-ES. *PPSN*.

[17] Shyama Prasad, P., Anto, M., Eshuijs, L., Moncarz, J., Kislay, K., & Vazquez, J. J. (2026). BaitBench: Measuring agent reward hacking with optional shortcuts planted in ML tasks. arXiv:2608.30724.

[18] Snoek, J., Larochelle, H., & Adams, R. P. (2012). Practical Bayesian optimization of machine learning algorithms. *NeurIPS*.

[19] Tavakoli, A., Pardo, F., & Kormushev, P. (2018). Action branching architectures for deep reinforcement learning. *AAAI*.

[20] van Hasselt, H., Guez, A., & Silver, D. (2016). Deep reinforcement learning with double Q-learning. *AAAI*.

[21] Volpp, M., Fröhlich, L. P., Fischer, K., Doerr, A., Falkner, S., Hutter, F., & Daniel, C. (2020). Meta-learning acquisition functions for transfer learning in Bayesian optimization. *ICLR*.

[22] Hansen, N., Finck, S., Ros, R., & Auger, A. (2009). Real-parameter black-box optimization benchmarking 2009: Noiseless functions definitions. INRIA Research Report RR-6829. https://hal-lara.archives-ouvertes.fr/inria-00362633v1

[23] Surjanovic, S., & Bingham, D. Virtual Library of Simulation Experiments: Optimization test problems. https://www.sfu.ca/~ssurjano/optimization.html

[24] Hansen, N., Akimoto, Y., & Baudis, P. pycma: Python implementation of CMA-ES. https://github.com/CMA-ES/pycma

[25] Rapin, J., & Teytaud, O. (2018). Nevergrad: A gradient-free optimization platform. https://github.com/facebookresearch/nevergrad